

Final Project: Regression and Classification with Neural Networks

Enock Owusu (22688016)

Part 0: Data Preparation and Exploratory Analysis

This section of the project looks at regression and classification tasks using multiple neural network architectures including DNN, CNN, LSTM, and hybrid models. Also data preparation and exploratory analysis was also conducted.

2.1 Jena Climate Dataset

```
'data.frame':  420551 obs. of  15 variables:
 $ Date.Time      : chr  "01.01.2009 00:10:00" "01.01.2009 00:20:00" "01.01.2009 00:30:00" "
 $ p..mbar.       : num  997 997 997 997 997 ...
 $ T..degC.       : num  -8.02 -8.41 -8.51 -8.31 -8.27 -8.05 -7.62 -7.62 -
7.91 -8.43 ...
 $ Tpot..K.       : num  265 265 265 265 265 ...
 $ Tdew..degC.    : num  -8.9 -9.28 -9.31 -9.07 -9.04 -8.78 -8.3 -8.36 -
8.73 -9.34 ...
 $ rh....        : num  93.3 93.4 93.9 94.2 94.1 94.4 94.8 94.4 93.8 93.1 ...
 $ VPmax..mbar.   : num  3.33 3.23 3.21 3.26 3.27 3.33 3.44 3.44 3.36 3.23 ...
 $ VPact..mbar.   : num  3.11 3.02 3.01 3.07 3.08 3.14 3.26 3.25 3.15 3 ...
 $ VPdef..mbar.   : num  0.22 0.21 0.2 0.19 0.19 0.19 0.18 0.19 0.21 0.22 ...
 $ sh..g.kg.      : num  1.94 1.89 1.88 1.92 1.92 1.96 2.04 2.03 1.97 1.88 ...
 $ H2OC..mmol.mol.: num  3.12 3.03 3.02 3.08 3.09 3.15 3.27 3.26 3.16 3.02 ...
 $ rho..g.m..3.   : num  1308 1310 1310 1309 1309 ...
 $ wv..m.s.       : num  1.03 0.72 0.19 0.34 0.32 0.21 0.18 0.19 0.28 0.59 ...
 $ max..wv..m.s.  : num  1.75 1.5 0.63 0.5 0.63 0.63 0.63 0.5 0.75 0.88 ...
 $ wd..deg.       : num  152 136 172 198 214 ...
```

	Date.Time	p..mbar.	T..degC.	Tpot..K.	Tdew..degC.	rh....
1	01.01.2009 00:10:00	996.52	-8.02	265.40	-8.90	93.3
2	01.01.2009 00:20:00	996.57	-8.41	265.01	-9.28	93.4
3	01.01.2009 00:30:00	996.53	-8.51	264.91	-9.31	93.9
4	01.01.2009 00:40:00	996.51	-8.31	265.12	-9.07	94.2
5	01.01.2009 00:50:00	996.51	-8.27	265.15	-9.04	94.1
6	01.01.2009 01:00:00	996.50	-8.05	265.38	-8.78	94.4

	VPmax..mbar.	VPact..mbar.	VPdef..mbar.	sh..g.kg.	H2OC..mmol.mol.	rho..g.m..3.
1	3.33	3.11	0.22	1.94	3.12	1307.75
2	3.23	3.02	0.21	1.89	3.03	1309.80
3	3.21	3.01	0.20	1.88	3.02	1310.24
4	3.26	3.07	0.19	1.92	3.08	1309.19
5	3.27	3.08	0.19	1.92	3.09	1309.00
6	3.33	3.14	0.19	1.96	3.15	1307.86

	wv..m.s.	max..wv..m.s.	wd..deg.
1	1.03	1.75	152.3
2	0.72	1.50	136.1
3	0.19	0.63	171.6
4	0.34	0.50	198.0
5	0.32	0.63	214.3
6	0.21	0.63	192.7

[1] 420551

[1] 70041

A tibble: 6 x 15

	Date.Hour	p..mbar.	T..degC.	Tpot..K.	Tdew..degC.	rh....	VPmax..mbar.
	<dtm>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2009-01-01 00:00:00	997.	-8.30	265.	-9.12	93.8	3.26
2	2009-01-01 01:00:00	997.	-8.06	265.	-8.86	93.9	3.32
3	2009-01-01 02:00:00	997.	-8.76	265.	-9.61	93.5	3.14
4	2009-01-01 03:00:00	997.	-8.90	264.	-9.79	93.2	3.11
5	2009-01-01 04:00:00	997.	-9.35	264.	-10.3	92.4	3.00
6	2009-01-01 05:00:00	998.	-9.72	264.	-10.7	92.4	2.92

i 8 more variables: VPact..mbar. <dbl>, VPdef..mbar. <dbl>, sh..g.kg. <dbl>,
H2OC..mmol.mol. <dbl>, rho..g.m..3. <dbl>, wv..m.s. <dbl>,
max..wv..m.s. <dbl>, wd..deg. <dbl>

tibble [70,041 x 15] (S3: tbl_df/tbl/data.frame)

\$ Date.Hour : POSIXct[1:70041], format: "2009-01-01 00:00:00" "2009-01-01 01:00:00" ...

```

$ p..mbar.      : num [1:70041] 997 997 997 997 997 ...
$ T..degC.      : num [1:70041] -8.3 -8.06 -8.76 -8.9 -9.35 ...
$ Tpot..K.      : num [1:70041] 265 265 265 264 264 ...
$ Tdew..degC.   : num [1:70041] -9.12 -8.86 -9.61 -9.79 -10.34 ...
$ rh....       : num [1:70041] 93.8 93.9 93.5 93.2 92.4 ...
$ VPmax..mbar.  : num [1:70041] 3.26 3.32 3.14 3.11 3 ...
$ VPact..mbar.  : num [1:70041] 3.06 3.12 2.94 2.9 2.77 ...
$ VPdef..mbar.  : num [1:70041] 0.202 0.202 0.202 0.21 0.232 ...
$ sh..g.kg.     : num [1:70041] 1.91 1.95 1.84 1.81 1.73 ...
$ H2OC..mmol.mol.: num [1:70041] 3.07 3.13 2.95 2.91 2.78 ...
$ rho..g.m..3.  : num [1:70041] 1309 1308 1312 1313 1315 ...
$ wv..m.s.      : num [1:70041] 0.52 0.317 0.248 0.177 0.29 ...
$ max..wv..m.s. : num [1:70041] 1.002 0.712 0.607 0.607 0.67 ...
$ wd..deg.      : num [1:70041] 174 172 197 157 150 ...

```

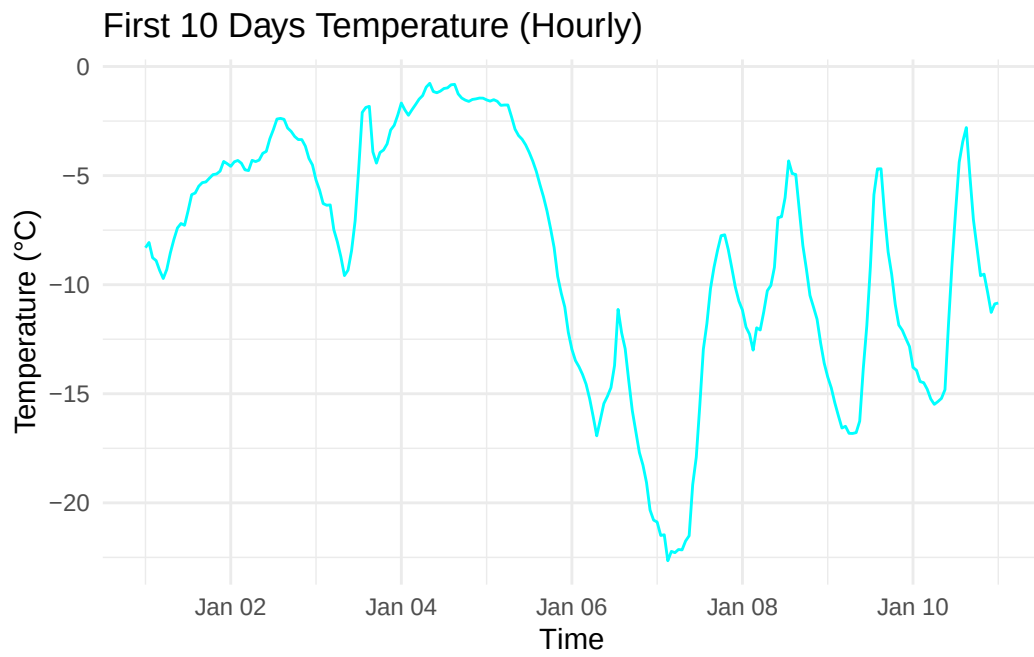
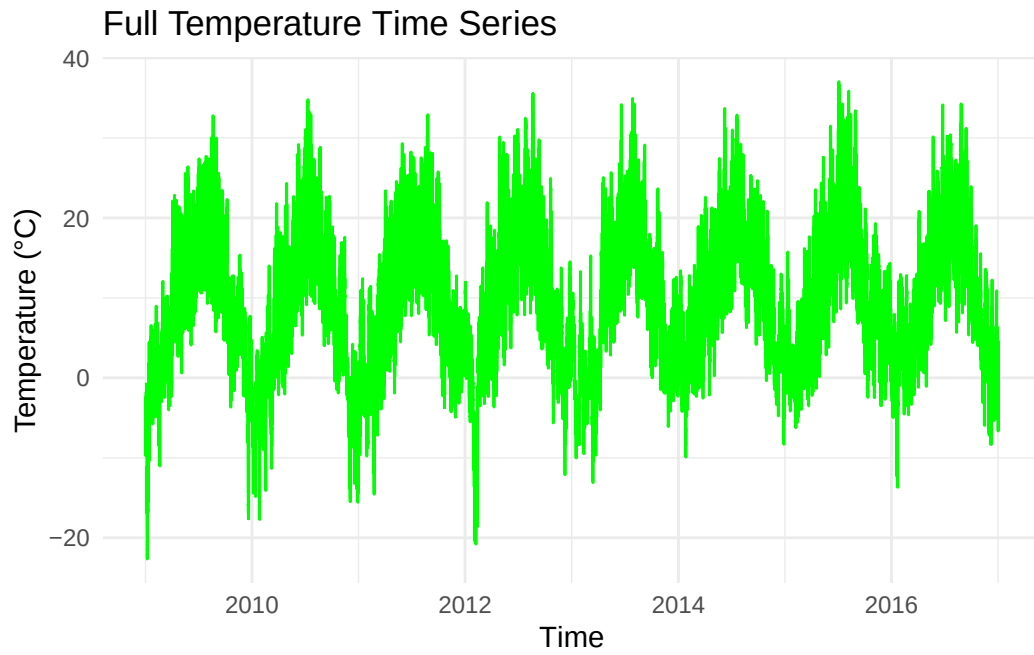
Brief Description

The Jena Climate dataset was imported from the given CSV file, and then the Date Time column was properly transformed to a datetime format using the `lubridate` library. The original dataset contained measurements made at every ten minutes. In order to adapt the data for the modelling purposes, the resampling of the data was done by changing the observation frequency from 10 minutes to one hour where the average value of all numeric variables for the respective hour was calculated. The data was converted to hourly observations because the models were trained using a 120-hour look-back window. There were 420551 observations before performing the resampling procedure; however, after resampling there remained 70041 observations only.

Date.Hour	p..mbar.	T..degC.	Tpot..K.	Tdew..degC.
0	0	0	0	0
rh....	VPmax..mbar.	VPact..mbar.	VPdef..mbar.	sh..g.kg.
0	0	0	0	0
H2OC..mmol.mol.	rho..g.m..3.	wv..m.s.	max..wv..m.s.	wd..deg.
0	0	0	0	0

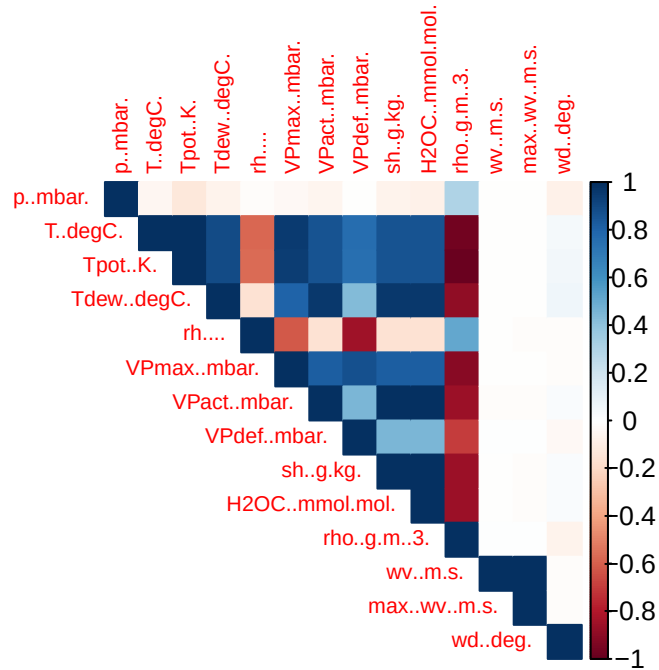
Missing Values Analysis

Missing values in the dataset were tested by finding the total number of missing observations per variable in the dataset. It is obvious that there are no missing values of significance exist within all the variables after the resampling process done on an hourly basis.



Time Series Visualization

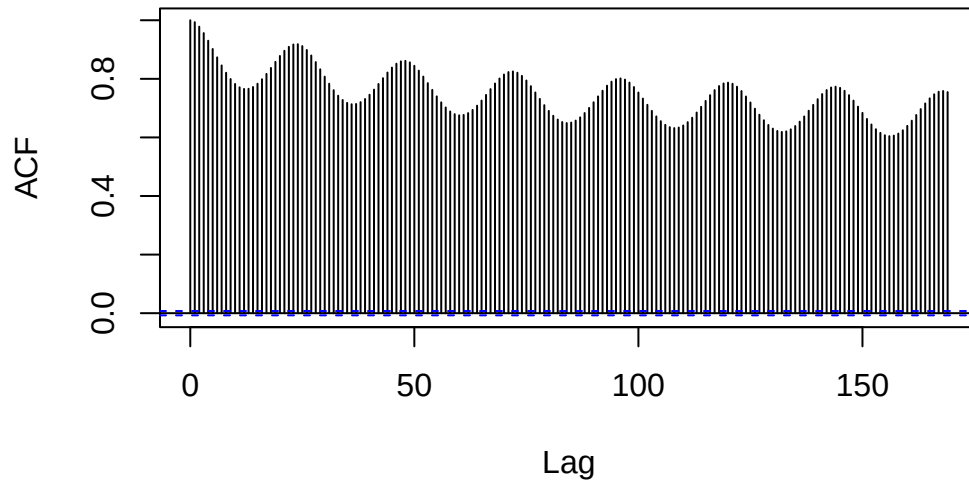
The complete temperature data shows definite trends in terms of seasons. There is a periodic rise and fall in temperatures every year, which clearly indicates that there is an underlying seasonality in temperature trends. This is evident by the graph plotted for the first ten days' temperatures measured per hour, which shows a definite diurnal trend. This trend shows a definite increase in temperatures during the day and fall during the night. This shows that there is significant variation within the day as well. Seasonality and diurnal trends make it essential to choose 120 hours as the look-back period.



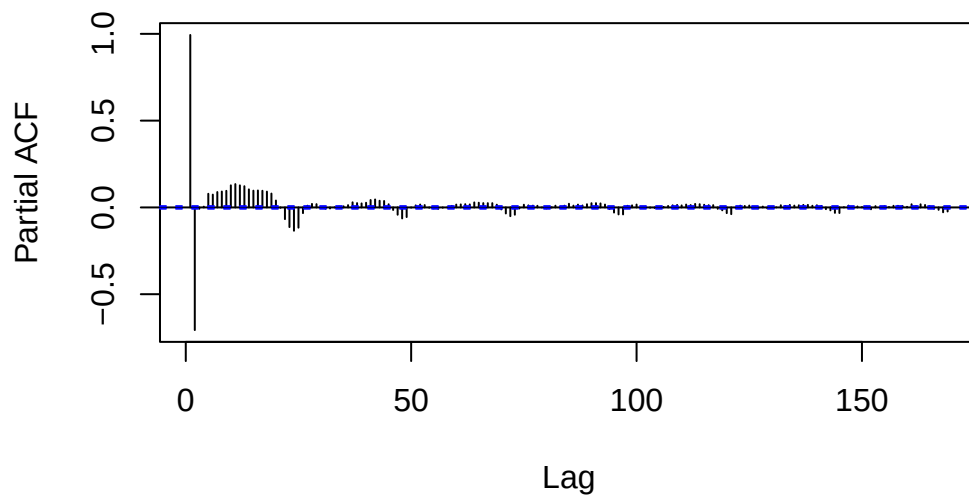
Correlation Analysis and Multicollinearity

In the correlation matrix, the relationship between all predictor variables included in the data set is highlighted. Many variables demonstrate a high positive or negative correlation, especially when comparing variables related to temperature, such as `T..degC.`, `Tpot..K.`, `Tdew..degC.`, and those related to vapour pressure. In turn, this suggests that there is a problem of multicollinearity in which predictors provide redundant data.

ACF of Hourly Temperature Series

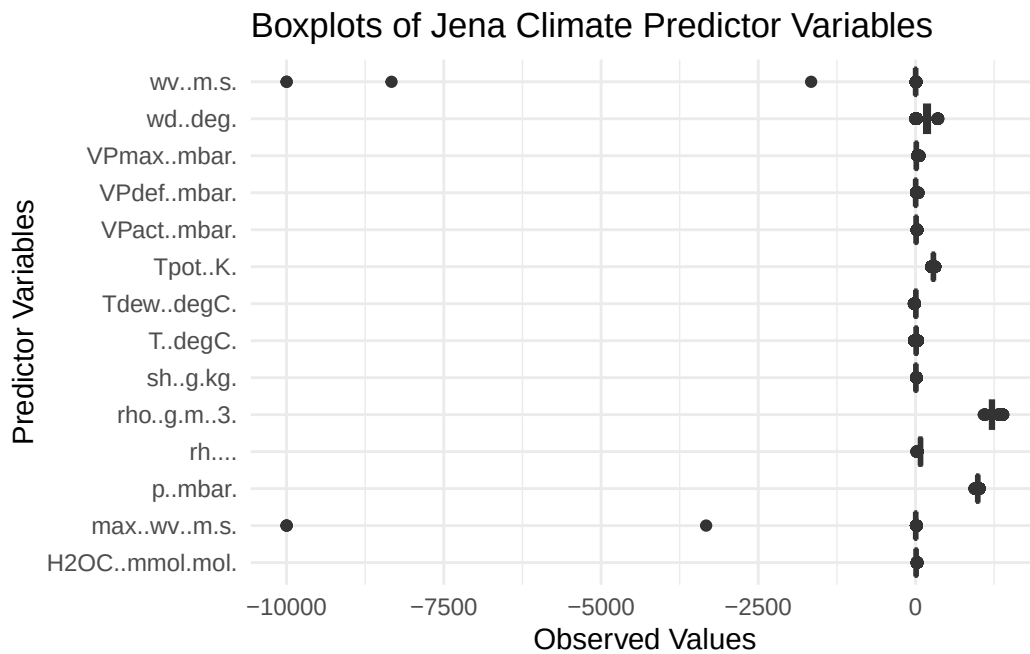


PACF of Hourly Temperature Series



ACF and PACF Interpretation

The ACF plot shows that temperature values are highly correlated across many previous hours. The slow decay in the autocorrelation function suggests strong temporal dependence in the series. This is expected because current temperature is influenced by recent past temperatures. The PACF plot also shows important partial correlations at earlier lags, indicating that past hourly observations contain useful predictive information. The use of a 120-hour lookback window is therefore appropriate because it allows the model to use five days of historical weather information to predict temperature 24 hours ahead. This window is long enough to capture short-term hourly variation, daily cycles, and broader multi-day temperature patterns.



Predictor Range and Need for Normalization

The box plots indicate that the predictor variables are highly diverse in terms of their scales of measurement. Predictor variables include those related to atmospheric pressure, temperature, humidity, wind direction, and vapor pressure, among others, which differ from each other in terms of their units and numerical scale. This validates the necessity of performing normalization before applying the neural network model. The reason for doing so is that neural networks are affected by the magnitude of the predictor variables.

p..mbar.	T..degC.	Tpot..K.	Tdew..degC.
Min. :-6.57247	Min. :-3.73229	Min. :-3.7544	Min. :-4.24662

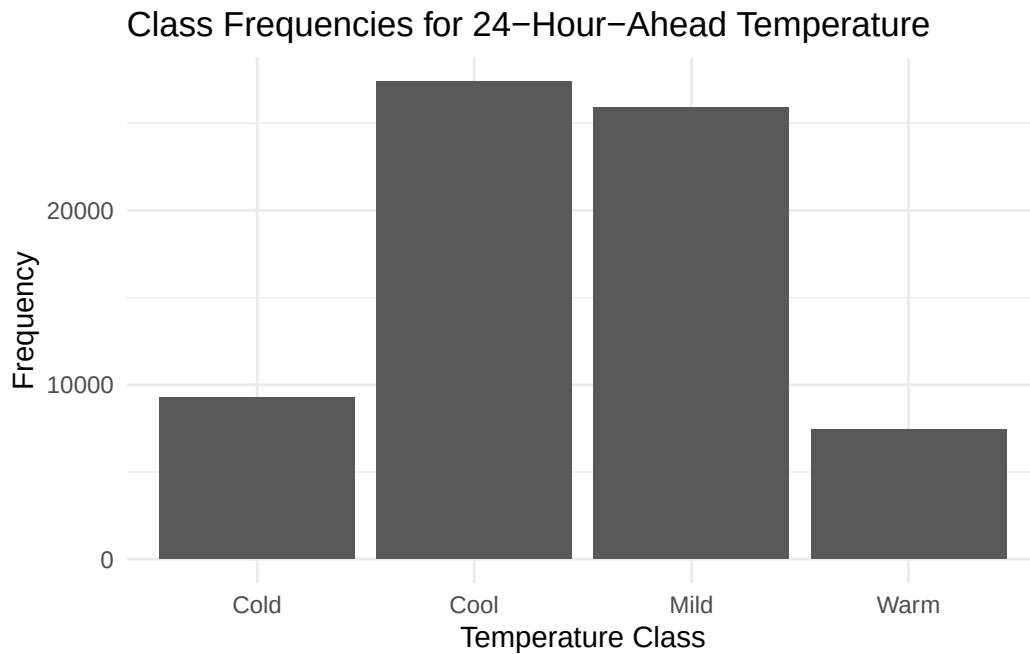
1st Qu.: -0.60791	1st Qu.: -0.71130	1st Qu.: -0.6980	1st Qu.: -0.67247
Median : 0.05157	Median : 0.01673	Median : 0.0201	Median : 0.06105
Mean : 0.00000	Mean : 0.00000	Mean : 0.0000	Mean : 0.00000
3rd Qu.: 0.65303	3rd Qu.: 0.71701	3rd Qu.: 0.7103	3rd Qu.: 0.75844
Max. : 2.93737	Max. : 3.09010	Max. : 3.0771	Max. : 2.63597
rh....	VPmax..mbar.	VPact..mbar.	VPdef..mbar.
Min. : -3.8361	Min. : -1.6367	Min. : -2.0573	Min. : -0.8284
1st Qu.: -0.6474	1st Qu.: -0.7561	1st Qu.: -0.7844	1st Qu.: -0.6539
Median : 0.2022	Median : -0.2134	Median : -0.1441	Median : -0.3820
Mean : 0.0000	Mean : 0.0000	Mean : 0.0000	Mean : 0.0000
3rd Qu.: 0.8094	3rd Qu.: 0.5347	3rd Qu.: 0.6809	3rd Qu.: 0.2639
Max. : 1.4400	Max. : 5.8945	Max. : 4.4569	Max. : 8.0527
sh..g.kg.	H2OC..mmol.mol.	rho..g.m..3.	wv..m.s.
Min. : -2.0493	Min. : -2.0567	Min. : -3.27470	Min. : -1.4719
1st Qu.: -0.7827	1st Qu.: -0.7828	1st Qu.: -0.70973	1st Qu.: -0.7481
Median : -0.1422	Median : -0.1407	Median : -0.08129	Median : -0.2373
Mean : 0.0000	Mean : 0.0000	Mean : 0.00000	Mean : 0.0000
3rd Qu.: 0.6772	3rd Qu.: 0.6786	3rd Qu.: 0.64999	3rd Qu.: 0.4819
Max. : 4.5215	Max. : 4.4933	Max. : 4.34120	Max. : 7.3890
max..wv..m.s.	wd..deg.		
Min. : -1.6051	Min. : -2.3497		
1st Qu.: -0.7688	1st Qu.: -0.4690		
Median : -0.2244	Median : 0.2402		
Mean : 0.0000	Mean : 0.0000		
3rd Qu.: 0.5332	3rd Qu.: 0.6797		
Max. : 7.4754	Max. : 2.4683		

Z-score Normalization

Z-score normalization was applied to all numeric variables using only the mean and standard deviation computed from the training set. The validation and test sets were then transformed using the same training-set statistics. This is important because validation and test data must represent unseen future observations. If their means and standard deviations are used during scaling, information from the future would leak into the training process.

```
# A tibble: 4 x 3
  temp_class    n percentage
  <chr>      <int>      <dbl>
1 Cold       9274       13.2
2 Cool      27376       39.1
3 Mild      25918       37.0
```


4 Warm 7449 10.6



A Derived Temperature Classification Label

A four-class classification target was created from the 24-hour-ahead temperature. Temperatures below 0°C were classified as Cold, temperatures from 0°C to 9.9°C as Cool, temperatures from 10°C to 19.9°C as Mild, and temperatures of 20°C or above as Warm. The class frequency table above shows the distribution of the four temperature categories.

```
[1] 128 120 14
```

NULL

Data Generator Function

A custom data generator function was developed to construct input-output sequences suitable for neural network training. The function accepts parameters including lookback (120 hours), delay (24 hours), step size, batch size, shuffle option, and task type (regression or classification). For each sample, the generator extracts a sequence of past observations over a 120-hour window

and pairs it with a target value corresponding to the temperature 24 hours ahead. This sliding window approach transforms the time-series data into a supervised learning format. For regression tasks, the target is the continuous temperature value. For classification tasks, the target is the categorical temperature class (Cold, Cool, Mild, Warm), encoded appropriately. The generated input data has a three-dimensional structure: (batch size, time steps, number of features), which is suitable for deep learning architectures such as DNN, CNN, and LSTM.

```
[1] 60000    28    28
```

```
[1] 10000    28    28
```

```
[1]    0 255
```

```
[1]    0 255
```

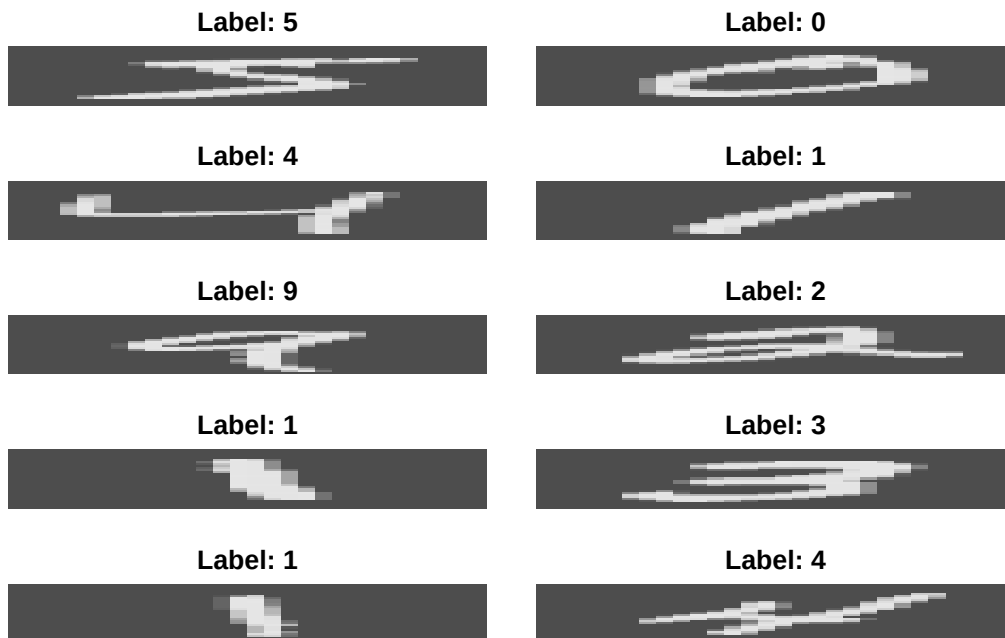
0.B MNIST

MNIST Handwritten Digit Dataset was imported by invoking the function “dataset_mnist()” in Keras library. The training dataset comprises 60,000 grayscale images, while the test dataset comprises 10,000 images. Every image is of size 28×28 pixels. The pixel intensity lies between 0 and 255.

```
[1] 50000    28    28
```

```
[1] 10000    28    28
```

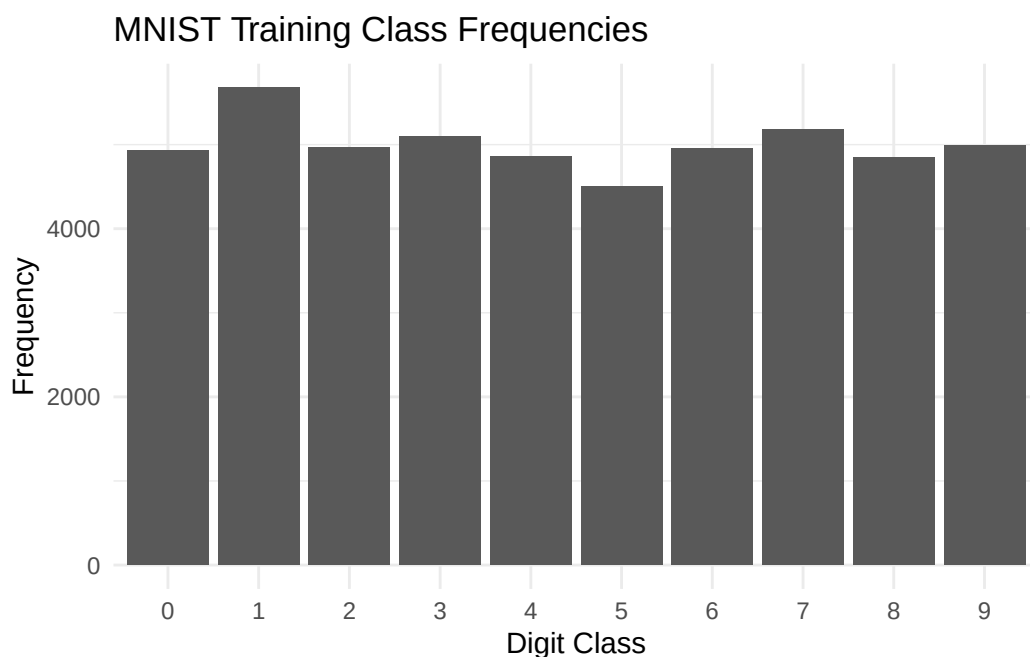
```
[1] 10000    28    28
```



Sample MNIST Images

Five by two grids of sample images for the MNIST dataset were shown alongside their respective label numbers. These images consist of handwritten digits ranging from 0 through 9 presented in grayscale images.

	Digit	Frequency	Percentage
1	0	4932	9.86
2	1	5678	11.36
3	2	4968	9.94
4	3	5101	10.20
5	4	4859	9.72
6	5	4506	9.01
7	6	4951	9.90
8	7	5175	10.35
9	8	4842	9.68
10	9	4988	9.98



```
[1] 50000    784
```

```
[1] 50000     28     28     1
```

```
[1] 50000     10
```

Input Transformations for Neural Networks

For the DNN baseline, each 28×28 image was reshaped into a 784-dimensional vector. This format is suitable for fully connected dense layers, which require one-dimensional input features. For the CNN model, the original spatial image structure was preserved by reshaping each image into a $28 \times 28 \times 1$ tensor. The final dimension represents the single grayscale channel. This format allows convolutional layers to learn local spatial patterns such as edges, curves, and digit strokes. The digit labels were also converted into one-hot encoded vectors with 10 classes, which is required when using categorical cross-entropy loss with a softmax output layer.

Part I: Vanilla Deep Neural Network

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 1680)	0
dense (Dense)	(None, 128)	215,168
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 1)	65

Total params: 223,489 (873.00 KB)

Trainable params: 223,489 (873.00 KB)

Non-trainable params: 0 (0.00 B)

Regression DNN Architecture

A simple deep neural network architecture has been chosen to serve as a base model for forecasting temperatures in Jena. As there are 120 samples of data per hour for each predictive variable, the input was converted from multi-dimensional to one-dimensional and fed into densely connected layers. Two hidden layers using rectified linear unit (ReLU) activation have been utilised in the proposed model. ReLU was chosen as an activation function since it brings nonlinearity into a neural network while mitigating the problem of vanishing gradients in contrast to sigmoid or hyperbolic tangent (tanh). There are 128 neurones in the first hidden layer and 64 neurones in the second hidden layer. One linearly activated neurone is contained in the output layer. Linear activation makes sense for regression, as the dependent variable, 24-hour-ahead temperature, is not bounded and can be either positive or negative.

Design a DNN

DNN Without Regularization

Model: "sequential_1"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 1680)	0

dense_3 (Dense)	(None, 128)	215,168
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 1)	65

Total params: 223,489 (873.00 KB)
 Trainable params: 223,489 (873.00 KB)
 Non-trainable params: 0 (0.00 B)

[1] 223489

Epoch 1/50

200/200 - 25s - 124ms/step - loss: 0.3989 - mae: 0.3989 - val_loss: 0.6932 - val_mae: 0.6932

Epoch 2/50

200/200 - 19s - 97ms/step - loss: 0.2995 - mae: 0.2995 - val_loss: 0.5073 - val_mae: 0.5073

Epoch 3/50

200/200 - 20s - 101ms/step - loss: 0.2808 - mae: 0.2808 - val_loss: 0.3835 - val_mae: 0.3835

Epoch 4/50

200/200 - 17s - 84ms/step - loss: 0.2675 - mae: 0.2675 - val_loss: 0.3257 - val_mae: 0.3257

Epoch 5/50

200/200 - 16s - 79ms/step - loss: 0.2603 - mae: 0.2603 - val_loss: 0.3530 - val_mae: 0.3530

Epoch 6/50

200/200 - 18s - 91ms/step - loss: 0.2541 - mae: 0.2541 - val_loss: 0.3333 - val_mae: 0.3333

Epoch 7/50

200/200 - 17s - 84ms/step - loss: 0.2495 - mae: 0.2495 - val_loss: 0.4071 - val_mae: 0.4071

Epoch 8/50

200/200 - 18s - 89ms/step - loss: 0.2398 - mae: 0.2398 - val_loss: 0.3988 - val_mae: 0.3988

Epoch 9/50

200/200 - 17s - 84ms/step - loss: 0.2371 - mae: 0.2371 - val_loss: 0.3357 - val_mae: 0.3357

DNN with Dropout

Model: "sequential_2"

Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 1680)	0
dense_6 (Dense)	(None, 128)	215,168

dropout (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_8 (Dense)	(None, 1)	65

Total params: 223,489 (873.00 KB)

Trainable params: 223,489 (873.00 KB)

Non-trainable params: 0 (0.00 B)

[1] 223489

Epoch 1/50

200/200 - 20s - 100ms/step - loss: 0.5233 - mae: 0.5233 - val_loss: 0.3785 - val_mae: 0.3785

Epoch 2/50

200/200 - 20s - 98ms/step - loss: 0.3668 - mae: 0.3668 - val_loss: 0.3739 - val_mae: 0.3739

Epoch 3/50

200/200 - 16s - 78ms/step - loss: 0.3441 - mae: 0.3441 - val_loss: 0.3540 - val_mae: 0.3540

Epoch 4/50

200/200 - 17s - 86ms/step - loss: 0.3336 - mae: 0.3336 - val_loss: 0.3699 - val_mae: 0.3699

Epoch 5/50

200/200 - 21s - 105ms/step - loss: 0.3267 - mae: 0.3267 - val_loss: 0.4115 - val_mae: 0.4115

DNN with L2 Weight Decay

Model: "sequential_3"

Layer (type)	Output Shape	Param #
flatten_3 (Flatten)	(None, 1680)	0
dense_9 (Dense)	(None, 128)	215,168
dense_10 (Dense)	(None, 64)	8,256
dense_11 (Dense)	(None, 1)	65

Total params: 223,489 (873.00 KB)

Trainable params: 223,489 (873.00 KB)
Non-trainable params: 0 (0.00 B)

[1] 223489

Epoch 1/50

200/200 - 24s - 121ms/step - loss: 0.6837 - mae: 0.3814 - val_loss: 0.6188 - val_mae: 0.3409

Epoch 2/50

200/200 - 27s - 137ms/step - loss: 0.5533 - mae: 0.2997 - val_loss: 0.5872 - val_mae: 0.3573

Epoch 3/50

200/200 - 19s - 93ms/step - loss: 0.4931 - mae: 0.2846 - val_loss: 0.7227 - val_mae: 0.5346

Epoch 4/50

200/200 - 21s - 103ms/step - loss: 0.4389 - mae: 0.2682 - val_loss: 0.6136 - val_mae: 0.4592

Epoch 5/50

200/200 - 15s - 76ms/step - loss: 0.4051 - mae: 0.2644 - val_loss: 0.4510 - val_mae: 0.3230

Epoch 6/50

200/200 - 16s - 78ms/step - loss: 0.3734 - mae: 0.2555 - val_loss: 0.3965 - val_mae: 0.2881

Epoch 7/50

200/200 - 17s - 84ms/step - loss: 0.3560 - mae: 0.2552 - val_loss: 0.5114 - val_mae: 0.4179

Epoch 8/50

200/200 - 15s - 75ms/step - loss: 0.3376 - mae: 0.2494 - val_loss: 0.3944 - val_mae: 0.3112

Epoch 9/50

200/200 - 22s - 109ms/step - loss: 0.3277 - mae: 0.2482 - val_loss: 0.4697 - val_mae: 0.3940

Epoch 10/50

200/200 - 23s - 116ms/step - loss: 0.3174 - mae: 0.2447 - val_loss: 0.3643 - val_mae: 0.2942

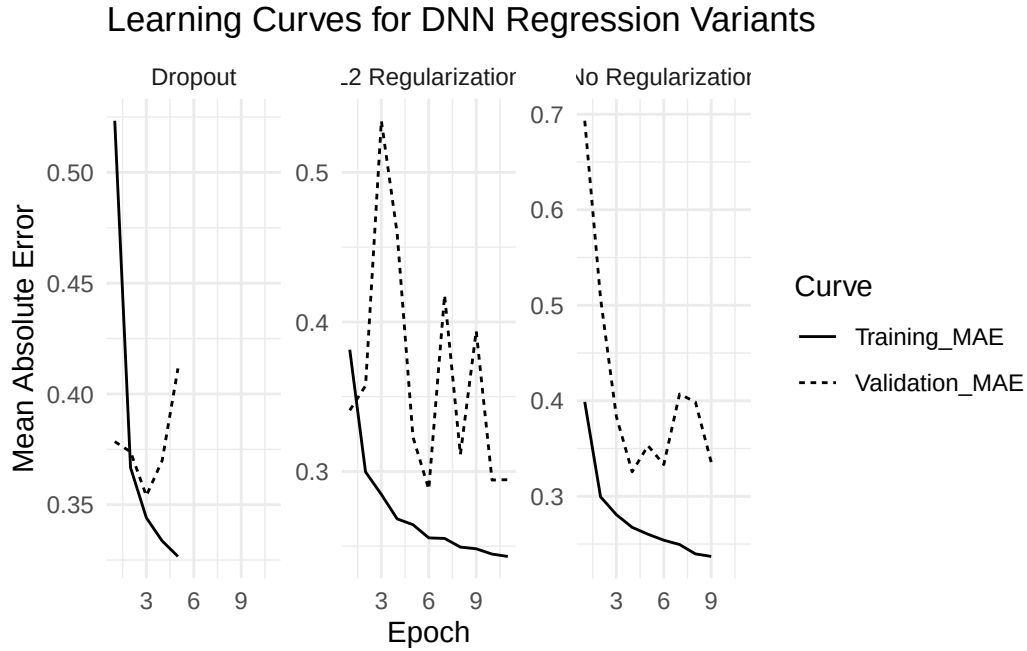
Epoch 11/50

200/200 - 19s - 93ms/step - loss: 0.3108 - mae: 0.2431 - val_loss: 0.3601 - val_mae: 0.2945

Parameter Count Table

	Model	Trainable_Parameters
1	DNN No Regularization	223489
2	DNN Dropout	223489
3	DNN L2 Regularization	223489

Learning Curves for the Three DNN Variants



Training of DNN Regression Models

Training of the three regression models was done through the Adam optimiser and loss function consisting of the mean absolute error. MAE loss was preferred since it represents the mean difference between the predictions and observations, expressed in degrees Celsius. In training the models, early stopping with patience of 5 epochs based on validation MAE was performed. That is, training ended automatically when the model failed to make improvements in validation MAE for five consecutive epochs. Moreover, the best model parameters were restored at the end of the training process to prevent an overfitted model from being retained. In all, three versions of the DNN (Deep Neural Network) were employed. For the first, no regularisation was used, representing the baseline model. The second model included a dropout technique for deactivating a portion of the neurones. The final model used weight decay through an L2 penalty. Learning curves were plotted to identify the bias/variance characteristics of each model. In cases where the no-regularisation model yields lower training MAE but higher validation MAE, then the model is said to be highly variegated due to overfitting. The dropout and L2 techniques are meant to address the issue of overfitting by enhancing regularisation.

Select the Best DNN Variant

No_Regularization	Dropout	L2_Regularization
0.3256886	0.3540132	0.2880946

```
[1] "L2_Regularization"
```

Creating Test Arrays For Evaluation

```
[1] 8567 120 14
```

```
[1] 8567
```

Predictions From the Best DNN Model

268/268 - 2s - 8ms/step

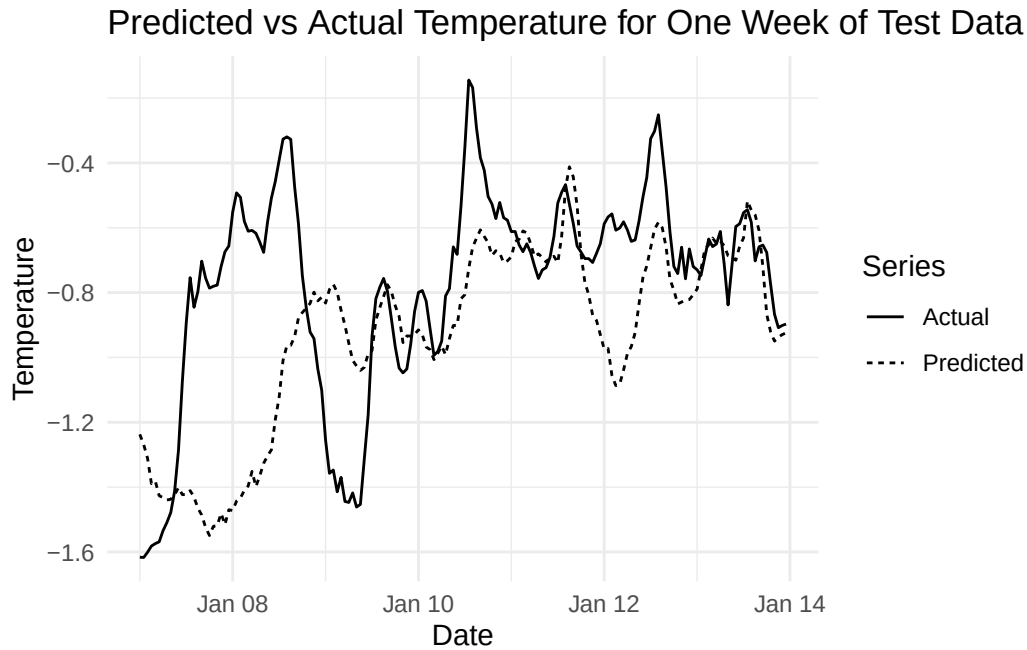
Test Metrics

	Best_Model	Test_MAE	Test_RMSE	Test_R2
1	L2_Regularization	0.3051	0.3837	0.8386

Naive Baseline: Predict $T+24 = T+0$

	Model	Test_MAE
1	Naive Baseline	0.2896
2	Best DNN: L2_Regularization	0.3051

One-Week Predicted vs Actual Plot



Test Evaluation of the Best DNN Variant

Selection of the best-performing model relied on the minimum MAE value of the validation samples. As a result, the final model is chosen with regard for its ability to perform well on unseen validation data.

The best DNN performed with an MAE of 0.3051, RMSE of 0.3837 and R^2 of 0.8386 on the test set. The MAE represents the mean of the absolute differences between the predicted values and the target variable in temperature units. RMSE is used to punish models for making predictions that differ considerably from actual results. Finally, R^2 is the share of the 24-hour-ahead temperature variability explained by the model.

A visual interpretation of the predictive power of the model can be provided with the use of a one-week prediction plot. It shows the accuracy of the predictions compared to the changes in the temperature.

Naive baseline predicts 24-hour-ahead temperature by taking into account the temperature at the moment of prediction. Its MAE value equals to 0.2896. A comparison of the MAE values of naive baseline and the best-performing DNN allows concluding about the model's predictive value. In case the MAE of the DNN is lower, it means that a prediction model provides additional insight.

2-D convolutional filter operates on a greyscale image versus 1-D filter operates on a multivariate time-series window

A 2-D convolutional filter operates on a greyscale image by moving across the image in two directions: height and width. At each position, the filter covers a small local patch of pixels, such as a 3×3 region, and computes a weighted sum of the pixel values. This produces a feature map that helps detect spatial patterns such as edges, curves, corners, and digit strokes. A 1-D convolutional filter operates differently because it moves along only one main direction: time. In a multivariate time-series window, the filter slides across consecutive time steps and examines short temporal segments.

inductive bias and parameter efficiency

Inductive bias shared by both 2D and 1D convolution models is locality in connections as well as parameter sharing. Locality of connections means that features in small neighbourhoods are more correlated than those far apart. In case of an image, the features within close proximity correspond to visual concepts, while, for time series, adjacent points tend to represent a particular pattern. The principle of parameter sharing allows applying the same kernel to different parts of the input data. Rather than training separate weights for all pixels or points, CNN trains a single filter and utilises it multiple times, which results in fewer parameters being trained when compared to traditional DNN.

Why local connectivity suits images and time series but is less natural for tabular data

Local connectivity is well-suited for image data since adjacent pixels belong to the same pattern most of the time. For instance, the strokes used to write a handwritten number are composed of pixels adjacent to each other. Thus, a local filter can extract useful features from them. Local connectivity is also well-suited for time-series data since values that are closer in time tend to be related. For instance, current temperatures are highly related to previous temperatures. Thus, a 1-D filter, which is a type of algorithm that processes data in a single dimension, is able to pick up the relations existing in a short period of time. However, local connectivity is not suited for tabular data because there isn't a spatial or temporal connection between variables in a tabular dataset.

II.B(a) 2D-CNN for MNIST Classification

Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	320
conv2d_1 (Conv2D)	(None, 28, 28, 64)	18,496
max_pooling2d (MaxPooling2D)	(None, 14, 14, 64)	0
flatten_4 (Flatten)	(None, 12544)	0
dense_12 (Dense)	(None, 128)	1,605,760
dense_13 (Dense)	(None, 10)	1,290

Total params: 1,625,866 (6.20 MB)

Trainable params: 1,625,866 (6.20 MB)

Non-trainable params: 0 (0.00 B)

Interpretation

The 2D-CNN architecture utilises the input shape (28, 28, 1) where the images are 28×28 in size and represent greyscale images with only one colour channel. The model has two Conv2D layers followed by rectified linear unit (ReLU) activation and same-padding. This allows for extracting spatially localised features such as edges, curves, and strokes from the digits. The MaxPooling2D layer allows for dimensionality reduction of the output but preserves important features from the previous layers. Lastly, the fully connected layer was used to classify the input data into one of the 10 classes using softmax activation.

Compile the CNN Model

Training Time per Epoch

Epoch 1/20

391/391 - 297s - 760ms/step - accuracy: 0.9480 - loss: 0.1750 - val_accuracy: 0.9751 - val_loss: 0.0489

Epoch 2/20

391/391 - 350s - 896ms/step - accuracy: 0.9847 - loss: 0.0489 - val_accuracy: 0.9852 - val_loss: 0.0290

Epoch 3/20

391/391 - 351s - 898ms/step - accuracy: 0.9908 - loss: 0.0290 - val_accuracy: 0.9868 - val_loss: 0.0290

```

Epoch 4/20
391/391 - 345s - 881ms/step - accuracy: 0.9938 - loss: 0.0196 - val_accuracy: 0.9860 - val_loss: 0.0212
Epoch 5/20
391/391 - 327s - 835ms/step - accuracy: 0.9956 - loss: 0.0134 - val_accuracy: 0.9885 - val_loss: 0.0212
Epoch 6/20
391/391 - 324s - 829ms/step - accuracy: 0.9960 - loss: 0.0112 - val_accuracy: 0.9858 - val_loss: 0.0212
Epoch 7/20
391/391 - 324s - 829ms/step - accuracy: 0.9972 - loss: 0.0079 - val_accuracy: 0.9870 - val_loss: 0.0212
Epoch 8/20
391/391 - 316s - 808ms/step - accuracy: 0.9979 - loss: 0.0059 - val_accuracy: 0.9895 - val_loss: 0.0212
Epoch 9/20
391/391 - 318s - 813ms/step - accuracy: 0.9985 - loss: 0.0046 - val_accuracy: 0.9890 - val_loss: 0.0212
Epoch 10/20
391/391 - 325s - 830ms/step - accuracy: 0.9978 - loss: 0.0063 - val_accuracy: 0.9889 - val_loss: 0.0212
Epoch 11/20
391/391 - 330s - 844ms/step - accuracy: 0.9985 - loss: 0.0047 - val_accuracy: 0.9889 - val_loss: 0.0212
Epoch 12/20
391/391 - 345s - 883ms/step - accuracy: 0.9986 - loss: 0.0043 - val_accuracy: 0.9876 - val_loss: 0.0212
Epoch 13/20
391/391 - 350s - 895ms/step - accuracy: 0.9989 - loss: 0.0037 - val_accuracy: 0.9894 - val_loss: 0.0212

```

```
[1] 331.4049
```

Test Accuracy

```
313/313 - 20s - 63ms/step - accuracy: 0.9886 - loss: 0.0450
```

```
$accuracy
```

```
[1] 0.9886
```

Predictions and Confusion Matrix

```
313/313 - 19s - 61ms/step
```

	Predicted									
Actual	0	1	2	3	4	5	6	7	8	9
0	973	1	0	1	1	0	3	0	1	0
1	0	1131	0	1	0	0	1	1	1	0
2	1	1	1014	2	0	0	3	10	1	0

3	0	0	2	1002	0	2	0	2	1	1
4	0	0	0	0	970	0	4	0	1	7
5	0	0	1	6	0	882	3	0	0	0
6	6	2	0	0	1	4	943	0	2	0
7	1	0	4	1	0	0	0	1020	1	1
8	2	0	3	3	0	0	0	4	959	3
9	1	1	0	3	2	3	0	6	1	992

Macro-F1 Function

	Model	Dataset	Test_Accuracy	Macro_F1	Train_Time_Per_Epoch_Seconds
1	2D-CNN	MNIST	0.9886	0.9885	331.4

Interpretation

The 2D-CNN was trained using the Adam optimizer and categorical cross-entropy loss, consistent with the DNN classification model in Part I. Early stopping with patience 5 was used to stop training when validation loss stopped improving. The model achieved a test accuracy of 0.9886 and a macro-F1 score of 0.9885. The confusion matrix shows the distribution of correct and incorrect predictions across the ten digit classes.

PARAMETER COUNT FUNCTION

```
[1] 1625866
```

1D-CNN for Jena Regression

```
Model: "sequential_5"
```

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 120, 32)	2,272
conv1d_1 (Conv1D)	(None, 120, 64)	10,304
max_pooling1d (MaxPooling1D)	(None, 60, 64)	0

global_average_pooling1d (GlobalAveragePooling1D)	(None, 64)	0
dense_14 (Dense)	(None, 64)	4,160
dense_15 (Dense)	(None, 1)	65

Total params: 16,801 (65.63 KB)
 Trainable params: 16,801 (65.63 KB)
 Non-trainable params: 0 (0.00 B)

Interpretaton

The architecture of the 1D-CNN was intended for inputs of size $120 \times N$, where the integer 120 refers to the last 120 hourly records, and the other dimension corresponds to the predictor variables of the atmospheric features. There are two layers of Conv1D layers activated by the ReLU function, followed by one MaxPooling1D layer, one Global Average Pooling 1D layer, and finally a single output neurone using a linear activation function since we are performing regression.

b. Justify kernel size and temporal receptive field

The Conv1D filters have a kernel size of 5. As Jena's dataset has been resampled to hourly data, this implies that each of the convolution kernels considers five hours' local temporal context. This ensures that the model can identify local weather patterns such as rising temperatures, cooling temperatures, humidity, and atmospheric pressure. Adding a second layer of Conv1D filters enables the model to consider a larger temporal window. The first layer extracts local five-hour windows, while the second layer fuses those patterns to identify more general temporal patterns. Consequently, while each convolution filter considers only five hours of temporal data, by stacking them together, the model is able to identify more complex patterns over a larger window of 120 hours. A kernel size of 5 was selected since this value is adequate to capture short-term weather variability without oversmoothing the time series data.

Compile the 1D-CNN

Train the 1D-CNN

Epoch 1/50

200/200 - 44s - 218ms/step - loss: 0.4404 - mae: 0.4404 - val_loss: 0.4054 - val_mae: 0.4054
Epoch 2/50
200/200 - 43s - 213ms/step - loss: 0.4215 - mae: 0.4215 - val_loss: 0.4049 - val_mae: 0.4049
Epoch 3/50
200/200 - 39s - 196ms/step - loss: 0.4110 - mae: 0.4110 - val_loss: 0.4013 - val_mae: 0.4013
Epoch 4/50
200/200 - 29s - 147ms/step - loss: 0.3595 - mae: 0.3595 - val_loss: 0.3855 - val_mae: 0.3855
Epoch 5/50
200/200 - 38s - 191ms/step - loss: 0.2985 - mae: 0.2985 - val_loss: 0.4969 - val_mae: 0.4969
Epoch 6/50
200/200 - 31s - 156ms/step - loss: 0.2773 - mae: 0.2773 - val_loss: 0.4798 - val_mae: 0.4798
Epoch 7/50
200/200 - 52s - 262ms/step - loss: 0.2704 - mae: 0.2704 - val_loss: 0.4331 - val_mae: 0.4331
Epoch 8/50
200/200 - 41s - 207ms/step - loss: 0.2696 - mae: 0.2696 - val_loss: 0.3711 - val_mae: 0.3711
Epoch 9/50
200/200 - 39s - 196ms/step - loss: 0.2664 - mae: 0.2664 - val_loss: 0.4261 - val_mae: 0.4261
Epoch 10/50
200/200 - 30s - 151ms/step - loss: 0.2602 - mae: 0.2602 - val_loss: 0.3908 - val_mae: 0.3908
Epoch 11/50
200/200 - 75s - 373ms/step - loss: 0.2576 - mae: 0.2576 - val_loss: 0.4214 - val_mae: 0.4214
Epoch 12/50
200/200 - 119s - 596ms/step - loss: 0.2572 - mae: 0.2572 - val_loss: 0.4075 - val_mae: 0.4075
Epoch 13/50
200/200 - 90s - 451ms/step - loss: 0.2509 - mae: 0.2509 - val_loss: 0.4945 - val_mae: 0.4945

TEST EVALUATION

268/268 - 7s - 26ms/step

	Model	Test_MAE	Test_RMSE	Test_R2	Parameters	Train_Time_Per_Epoch_Seconds
1	1D-CNN	0.2796	0.3594	0.8584	16801	51.94

Compare 1D-CNN with DNN Baseline

	Model	Test_MAE	Test_RMSE	Test_R2	Parameters
1	DNN Baseline	0.3051	0.3837	0.8386	223489
2	1D-CNN	0.2796	0.3594	0.8584	16801

	Train_Time_Per_Epoch_Seconds
1	NA

Interpretation

The 1D-CNN was trained using Adam optimization and MAE loss, following the same evaluation setting as the DNN regression baseline. On the Jena test set, the model achieved a MAE of 0.2796, RMSE of 0.3594, and R^2 of 0.8584. Compared with the DNN baseline, the 1D-CNN preserves the temporal structure of the 120-hour input window. The DNN flattens the time-series input and treats all lagged values as ordinary features, while the 1D-CNN applies filters across consecutive hours. Therefore, the 1D-CNN is theoretically better suited to capturing local temporal dependencies in climate data. If the 1D-CNN achieves a lower MAE and RMSE or a higher R^2 than the DNN, this indicates that exploiting temporal locality improves forecasting performance.

Part III: Recurrent Neural Network (LSTM)

a. Why a vanilla Simple RNN struggles with sequences of 120+ steps

The standard vanilla Simple Recurrent Neural Network has problems processing sequences that are too long, since it requires the passing of information through a recurrent function repeatedly at each time step. When back propagation through time occurs, the gradient gets multiplied repeatedly by recurrent weight matrices. In case these multiplications produce values less than one, then the gradient gets vanishingly small during back propagation.

b. How LSTM gates carry information over long horizons

The Long Short-Term Memory network solves the problem of vanishing gradients using the concept of cell state and three types of gates: the forget gate, the input gate, and the output gate. The function of the forget gate is to identify whether information stored previously in the cell state must be forgotten or kept. The input gate determines the amount of information to add to the cell state. The output gate regulates the amount of information that is transferred from the cell state to the hidden state for use in predictions. Cell state enables the movement of information through several time steps with fewer non-linear transformations. For this reason, it becomes easier for LSTM to remember critical information in the long term. The concept is beneficial in the Jena climate example because future temperature at 24 hours could depend not only on the immediate one hour but also on daily and multi-day patterns in the lookback window of 120 hours.

c. Why treating MNIST rows as time steps is valid but suboptimal

Treating MNIST images as sequences is valid because each image can be represented as 28 rows, where each row contains 28 pixel values. Under this representation, the LSTM processes the image row by row as a sequence of 28 time steps with 28 features at each step. However, this is a suboptimal use of LSTM because MNIST is primarily a spatial image problem, not a temporal sequence problem. The arrangement of pixels contains two-dimensional spatial structure, including local strokes, curves, and edges. A CNN is better suited to this structure because its filters move across both height and width and directly capture local spatial patterns. Using an LSTM for MNIST shows the importance of inductive bias. Even if an LSTM has enough capacity to classify digits, its sequential bias does not match the image structure as naturally as the spatial locality and parameter sharing bias of a CNN. Therefore, the CNN is expected to outperform the LSTM on MNIST classification.

LSTM Model for Jena Regression

Model: "sequential_6"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 120, 64)	20,224
lstm_1 (LSTM)	(None, 32)	12,416
dense_16 (Dense)	(None, 1)	33

Total params: 32,673 (127.63 KB)

Trainable params: 32,673 (127.63 KB)

Non-trainable params: 0 (0.00 B)

Interpretation

Input for the LSTM model was set to be of size (120, **number of predictors**) since the lookback period is of 120 hours. Two LSTM layers were added, and the first layer's return sequences attribute was set to True in order for the second LSTM layer to process the whole temporal information. Recurrent dropout was incorporated into the model to avoid overfitting by randomly dropping units from the recurrent neural network. Linear activation function was chosen for the output layer due to the regression problem.

b. Train and Evaluate LSTM(COMPILE LSTM MODEL)

TRAIN LSTM

Epoch 1/50

200/200 - 239s - 1s/step - loss: 0.3499 - mae: 0.3499 - val_loss: 0.4491 - val_mae: 0.4491

Epoch 2/50

200/200 - 172s - 859ms/step - loss: 0.3038 - mae: 0.3038 - val_loss: 0.3827 - val_mae: 0.3827

Epoch 3/50

200/200 - 174s - 872ms/step - loss: 0.2938 - mae: 0.2938 - val_loss: 0.3542 - val_mae: 0.3542

Epoch 4/50

200/200 - 150s - 751ms/step - loss: 0.2913 - mae: 0.2913 - val_loss: 0.3610 - val_mae: 0.3610

Epoch 5/50

200/200 - 133s - 663ms/step - loss: 0.2826 - mae: 0.2826 - val_loss: 0.3436 - val_mae: 0.3436

Epoch 6/50

200/200 - 180s - 901ms/step - loss: 0.2809 - mae: 0.2809 - val_loss: 0.3448 - val_mae: 0.3448

Epoch 7/50

200/200 - 142s - 710ms/step - loss: 0.2765 - mae: 0.2765 - val_loss: 0.3183 - val_mae: 0.3183

Epoch 8/50

200/200 - 155s - 775ms/step - loss: 0.2731 - mae: 0.2731 - val_loss: 0.3220 - val_mae: 0.3220

Epoch 9/50

200/200 - 123s - 616ms/step - loss: 0.2724 - mae: 0.2724 - val_loss: 0.3215 - val_mae: 0.3215

Epoch 10/50

200/200 - 111s - 554ms/step - loss: 0.2679 - mae: 0.2679 - val_loss: 0.3349 - val_mae: 0.3349

Epoch 11/50

200/200 - 110s - 552ms/step - loss: 0.2651 - mae: 0.2651 - val_loss: 0.3292 - val_mae: 0.3292

Epoch 12/50

200/200 - 106s - 529ms/step - loss: 0.2626 - mae: 0.2626 - val_loss: 0.3246 - val_mae: 0.3246

TEST EVALUATION

268/268 - 27s - 101ms/step

	Model	Test_MAE	Test_RMSE	Test_R2	Parameters	Train_Time_Per_Epoch_Seconds
1	LSTM	0.2707	0.3475	0.8676	32673	149.81

c. Ablation: SimpleRNN vs LSTM

Train the SimpleRNN

Epoch 1/50

200/200 - 53s - 263ms/step - loss: 0.3293 - mae: 0.3293 - val_loss: 0.4602 - val_mae: 0.4602

Epoch 2/50

200/200 - 39s - 193ms/step - loss: 0.3002 - mae: 0.3002 - val_loss: 0.4073 - val_mae: 0.4073

Epoch 3/50

200/200 - 37s - 185ms/step - loss: 0.2955 - mae: 0.2955 - val_loss: 0.4432 - val_mae: 0.4432

Epoch 4/50

200/200 - 39s - 194ms/step - loss: 0.2901 - mae: 0.2901 - val_loss: 0.4314 - val_mae: 0.4314

Epoch 5/50

200/200 - 37s - 187ms/step - loss: 0.2888 - mae: 0.2888 - val_loss: 0.4552 - val_mae: 0.4552

Test Performance of SimpleRNN

268/268 - 14s - 51ms/step

[1] 0.2893243

Interpretation LSTM vs Simple RNN for Jena Regression

The training process for the LSTM model was performed in the same way as for the DNN model. The test results show that the LSTM network reached the values of MAE, RMSE, and R^2 equal to 0.2707, and 0.8676, respectively. In order to find out how important the property of memory is for the LSTM, an ablation model based on the SimpleRNN with the same structure as the LSTM model was trained. The result obtained from the Simple RNN showed that the test MAE equals 0.2893. Thus, the reason for the better performance of the LSTM model is associated with overcoming the problem of the vanishing gradient that occurs while training the RNN. Infact, during training, when passing through many steps, the gradient loses its value and makes it difficult for the RNN model to learn some patterns in the sequence.

III.C LSTM for Jena Classification

CLASSIFICATION DATA GENERATOR

CREATE CLASSIFICATION GENERATORS

```
[1] 128 120 14
```

```
[1] 128 4
```

LSTM Model for Jena Classification

```
Model: "sequential_8"
```

Layer (type)	Output Shape	Param #
lstm_2 (LSTM)	(None, 120, 64)	20,224
lstm_3 (LSTM)	(None, 32)	12,416
dense_18 (Dense)	(None, 4)	132

```
Total params: 32,772 (128.02 KB)
```

```
Trainable params: 32,772 (128.02 KB)
```

```
Non-trainable params: 0 (0.00 B)
```

Interpretation

For Jena classification, a softmax layer with four neurones was used instead of the regression output layer. The first, second, third, and fourth neurones represent the categories cold, cool, mild, and “warm” temperatures, respectively. The softmax layer was suitable since its purpose was to transform the output of the model into probabilities summing up to one. The loss function was also switched from MAE to categorical cross-entropy.

b. Train and evaluate: accuracy, macro-F1, confusion matrix

TRAIN LSTM CLASSIFICATION MODEL

```
Epoch 1/50
200/200 - 125s - 625ms/step - accuracy: 0.5950 - loss: 0.9296 - val_accuracy: 0.3984 - val_loss: 0.8125
Epoch 2/50
200/200 - 113s - 564ms/step - accuracy: 0.6436 - loss: 0.8153 - val_accuracy: 0.8125 - val_loss: 0.7969
Epoch 3/50
200/200 - 101s - 505ms/step - accuracy: 0.6544 - loss: 0.7799 - val_accuracy: 0.7031 - val_loss: 0.7693
Epoch 4/50
200/200 - 94s - 471ms/step - accuracy: 0.6657 - loss: 0.7365 - val_accuracy: 0.7969 - val_loss: 0.8438
Epoch 5/50
200/200 - 94s - 472ms/step - accuracy: 0.6639 - loss: 0.7693 - val_accuracy: 0.8438 - val_loss: 0.8125
Epoch 6/50
200/200 - 106s - 528ms/step - accuracy: 0.6747 - loss: 0.7389 - val_accuracy: 0.8125 - val_loss: 0.6833
Epoch 7/50
200/200 - 96s - 482ms/step - accuracy: 0.6833 - loss: 0.7311 - val_accuracy: 0.8516 - val_loss: 0.7311
```

CREATE TEST CLASSIFICATION ARRAYS

TEST PREDICTION

```
267/267 - 17s - 62ms/step
```

ACCURACY, MACRO-F1, CONFUSION MATRIX

	Model	Dataset	Test_Accuracy	Macro_F1
1	LSTM	Jena 4-Class	0.6482	0.4344

	Predicted			
Actual	Cold	Cool	Mild	Warm
Cold	191	576	0	0
Cool	412	3006	252	0
Mild	8	643	2341	0
Warm	0	13	1102	0

Interpretation

The accuracy for the LSTM classification model was $\text{round}(\text{lstm_class_accuracy}, 4)$ while the macro F1-score was $\text{round}(\text{lstm_class_macro_f1}, 4)$. The confusion matrix displays the classification of the four temperature classes. It was likely that misclassifications will happen between classes that have nearby temperature ranges, such as Cool vs. Mild and Mild vs. Warm.

c. Discussion: Regression or classification advantage?

The LSTM model shows a greater advantage over the DNN in the case of the regression problem than over the classification problem. When dealing with a regression problem, the LSTM model predicts the temperature for the next 24 hours, hence making use of learned temporal relationships, daily cycles, and weather patterns over a period of 120 hours. On the other hand, the regression output is changed into four broad classes when using the classification model. As a result, valuable information is lost since temperatures near the boundary of each category could be classified into different classes despite having similar numerical values.

III.D LSTM for MNIST Sequential Image

```
[1] 50000      28      28
```

```
[1] 10000      28      28
```

```
[1] 10000      28      28
```

Interpretation

The images from the MNIST dataset were converted to have a series of 28 timesteps, with each timestep consisting of one row of pixels of size 28 x 28. Therefore, each image used for training had an input shape of 28 timesteps, with each timestep representing one row of pixels.

b. Design and train LSTM model

Model: "sequential_9"

Layer (type)	Output Shape	Param #
lstm_4 (LSTM)	(None, 64)	23,808
dense_19 (Dense)	(None, 128)	8,320
dense_20 (Dense)	(None, 10)	1,290

Total params: 33,418 (130.54 KB)

Trainable params: 33,418 (130.54 KB)

Non-trainable params: 0 (0.00 B)

COMPILE MNIST LSTM

TRAIN MNIST LSTM

Epoch 1/20

391/391 - 42s - 108ms/step - accuracy: 0.6747 - loss: 0.9522 - val_accuracy: 0.8880 - val_loss: 0.1455

Epoch 2/20

391/391 - 21s - 53ms/step - accuracy: 0.8900 - loss: 0.3430 - val_accuracy: 0.9489 - val_loss: 0.0929

Epoch 3/20

391/391 - 21s - 55ms/step - accuracy: 0.9293 - loss: 0.2245 - val_accuracy: 0.9605 - val_loss: 0.0859

Epoch 4/20

391/391 - 21s - 54ms/step - accuracy: 0.9457 - loss: 0.1759 - val_accuracy: 0.9681 - val_loss: 0.0859

Epoch 5/20

391/391 - 22s - 56ms/step - accuracy: 0.9545 - loss: 0.1455 - val_accuracy: 0.9733 - val_loss: 0.0859

Epoch 6/20

391/391 - 24s - 60ms/step - accuracy: 0.9608 - loss: 0.1280 - val_accuracy: 0.9764 - val_loss: 0.0859

Epoch 7/20

391/391 - 23s - 59ms/step - accuracy: 0.9651 - loss: 0.1144 - val_accuracy: 0.9802 - val_loss: 0.0859

Epoch 8/20

391/391 - 22s - 56ms/step - accuracy: 0.9676 - loss: 0.1032 - val_accuracy: 0.9802 - val_loss: 0.0859

Epoch 9/20

391/391 - 22s - 55ms/step - accuracy: 0.9705 - loss: 0.0929 - val_accuracy: 0.9780 - val_loss: 0.0859

Epoch 10/20

391/391 - 21s - 54ms/step - accuracy: 0.9731 - loss: 0.0859 - val_accuracy: 0.9822 - val_loss: 0.0859

```

Epoch 11/20
391/391 - 21s - 55ms/step - accuracy: 0.9745 - loss: 0.0811 - val_accuracy: 0.9850 - val_loss: 0.0430
Epoch 12/20
391/391 - 22s - 57ms/step - accuracy: 0.9764 - loss: 0.0743 - val_accuracy: 0.9854 - val_loss: 0.0430
Epoch 13/20
391/391 - 22s - 57ms/step - accuracy: 0.9779 - loss: 0.0695 - val_accuracy: 0.9845 - val_loss: 0.0430
Epoch 14/20
391/391 - 23s - 58ms/step - accuracy: 0.9791 - loss: 0.0667 - val_accuracy: 0.9853 - val_loss: 0.0430
Epoch 15/20
391/391 - 22s - 55ms/step - accuracy: 0.9811 - loss: 0.0600 - val_accuracy: 0.9873 - val_loss: 0.0430
Epoch 16/20
391/391 - 21s - 55ms/step - accuracy: 0.9819 - loss: 0.0584 - val_accuracy: 0.9867 - val_loss: 0.0430
Epoch 17/20
391/391 - 30s - 77ms/step - accuracy: 0.9815 - loss: 0.0574 - val_accuracy: 0.9876 - val_loss: 0.0430
Epoch 18/20
391/391 - 34s - 87ms/step - accuracy: 0.9827 - loss: 0.0551 - val_accuracy: 0.9883 - val_loss: 0.0430
Epoch 19/20
391/391 - 25s - 64ms/step - accuracy: 0.9832 - loss: 0.0512 - val_accuracy: 0.9878 - val_loss: 0.0430
Epoch 20/20
391/391 - 25s - 64ms/step - accuracy: 0.9844 - loss: 0.0481 - val_accuracy: 0.9865 - val_loss: 0.0430

```

c. Test accuracy and comparison with CNN

```
313/313 - 4s - 13ms/step - accuracy: 0.9859 - loss: 0.0430
```

```
$accuracy
[1] 0.9859
```

PARAMETER COUNT

```
[1] 33418
```

Compare LSTM with CNN

	Model	Test_Accuracy	Parameters	Train_Time_Per_Epoch_Seconds
1	2D-CNN	0.9886	1625866	331.40
2	LSTM Sequential Image	0.9859	33418	24.41

Interpretation

The comparison between the 2D-CNN and LSTM models shows that the 2D-CNN achieved slightly higher classification accuracy (98.81%) compared to the LSTM model (98.59%). This indicates that the CNN model performed marginally better in classifying the image dataset. However, the CNN model required significantly more parameters (1,625,866) and a longer training time per epoch (350.91 seconds), making it computationally more expensive. In contrast, the LSTM model used far fewer parameters (33,418) and trained much faster (36.73 seconds per epoch).

Part IV.1 Design Brief — Custom Hybrid Architecture

For this task, a hybrid CNN–LSTM architecture was used to predict temperatures in the Jena climate dataset. The model combines a 1D Convolutional Neural Network (CNN) with a Long Short-Term Memory (LSTM) network so that both short-term and long-term patterns in the weather data can be captured effectively. The motivation for using this hybrid approach comes from the limitations of the individual models. The baseline Deep Neural Network (DNN) treats the 120-hour input window as a single vector, which means the sequential nature of the data is ignored. Because of this, the model struggles to learn important local temporal patterns within the climate variables. To improve this, the CNN component was introduced to capture short-term patterns such as hourly changes in temperature, humidity, and pressure. While CNNs are very good at extracting these local features, they are less effective at learning relationships that span several days. This is where the LSTM component becomes useful. The LSTM is designed to learn long-term dependencies and sequential relationships in time-series data.

IV.1 Design Brief (Using Suggested Hybrid Pattern: CNN–LSTM)

The CNN–LSTM hybrid model used in this project combines 1D convolutional layers with LSTM layers for temperature prediction using the Jena climate dataset. This architecture was selected to address the limitations of using individual models such as DNN, CNN, or LSTM alone. The DNN model ignores the sequential nature of the data because it flattens the input before training, making it difficult to capture temporal relationships. The 1D-CNN improves upon this by learning short-term temporal patterns and local dependencies within the climate variables. However, CNNs may struggle to capture long-term dependencies across the full 120-hour look-back window. On the other hand, the LSTM network is effective at learning long-term temporal dependencies through its memory and gating mechanisms, but it may not be as efficient as CNNs in extracting short-term local patterns. By combining the two architectures, the hybrid CNN–LSTM model benefits from the strengths of both approaches. In this framework, the Conv1D layers first extract local temporal features, such as hourly changes in temperature and humidity. These extracted feature sequences are then passed to

the LSTM layer, which learns the long-term relationships and sequential dependencies within the data. As a result, the model incorporates two important inductive biases: locality through CNNs and long-term memory through LSTMs. The Jena climate dataset was well suited for evaluating this hybrid architecture because it contains both short-term weather fluctuations and longer-term temporal trends. Since the task involves forecasting temperature 24 hours ahead using the previous 120 hours of observations, the CNN-LSTM model is expected to effectively capture both local and sequential patterns in the climate data.

Part IV: Custom Hybrid Architecture

Model: "CNN_LSTM_Hybrid_Jena_Regression"

Layer (type)	Output Shape	Param #
Input_120_hours (InputLayer)	(None, 120, 14)	0
Conv1D_Local_Features_1 (Conv1D)	(None, 120, 32)	2,272
Conv1D_Local_Features_2 (Conv1D)	(None, 120, 64)	10,304
MaxPooling_Temporal_Reduction (MaxPooling1D)	(None, 60, 64)	0
LSTM_Long_Range_Dependencies (LSTM)	(None, 64)	33,024
Dense_Feature_Integration (Dense)	(None, 64)	4,160
Temperature_Output (Dense)	(None, 1)	65

Total params: 49,825 (194.63 KB)

Trainable params: 49,825 (194.63 KB)

Non-trainable params: 0 (0.00 B)

COMPILE HYBRID MODEL

TRAIN HYBRID MODEL

Epoch 1/50

200/200 - 103s - 516ms/step - loss: 0.3388 - mae: 0.3388 - val_loss: 0.4639 - val_mae: 0.4639
Epoch 2/50
200/200 - 65s - 323ms/step - loss: 0.2937 - mae: 0.2937 - val_loss: 0.4632 - val_mae: 0.4632
Epoch 3/50
200/200 - 78s - 388ms/step - loss: 0.2856 - mae: 0.2856 - val_loss: 0.4116 - val_mae: 0.4116
Epoch 4/50
200/200 - 57s - 287ms/step - loss: 0.2822 - mae: 0.2822 - val_loss: 0.4273 - val_mae: 0.4273
Epoch 5/50
200/200 - 58s - 288ms/step - loss: 0.2706 - mae: 0.2706 - val_loss: 0.3968 - val_mae: 0.3968
Epoch 6/50
200/200 - 63s - 315ms/step - loss: 0.2707 - mae: 0.2707 - val_loss: 0.3441 - val_mae: 0.3441
Epoch 7/50
200/200 - 62s - 309ms/step - loss: 0.2672 - mae: 0.2672 - val_loss: 0.3754 - val_mae: 0.3754
Epoch 8/50
200/200 - 61s - 305ms/step - loss: 0.2610 - mae: 0.2610 - val_loss: 0.3790 - val_mae: 0.3790
Epoch 9/50
200/200 - 55s - 274ms/step - loss: 0.2603 - mae: 0.2603 - val_loss: 0.4011 - val_mae: 0.4011
Epoch 10/50
200/200 - 53s - 264ms/step - loss: 0.2524 - mae: 0.2524 - val_loss: 0.3281 - val_mae: 0.3281
Epoch 11/50
200/200 - 52s - 260ms/step - loss: 0.2525 - mae: 0.2525 - val_loss: 0.3647 - val_mae: 0.3647
Epoch 12/50
200/200 - 54s - 271ms/step - loss: 0.2502 - mae: 0.2502 - val_loss: 0.3526 - val_mae: 0.3526
Epoch 13/50
200/200 - 51s - 256ms/step - loss: 0.2440 - mae: 0.2440 - val_loss: 0.3369 - val_mae: 0.3369
Epoch 14/50
200/200 - 50s - 250ms/step - loss: 0.2396 - mae: 0.2396 - val_loss: 0.3271 - val_mae: 0.3271
Epoch 15/50
200/200 - 51s - 257ms/step - loss: 0.2341 - mae: 0.2341 - val_loss: 0.3334 - val_mae: 0.3334
Epoch 16/50
200/200 - 50s - 252ms/step - loss: 0.2325 - mae: 0.2325 - val_loss: 0.2829 - val_mae: 0.2829
Epoch 17/50
200/200 - 53s - 263ms/step - loss: 0.2265 - mae: 0.2265 - val_loss: 0.3441 - val_mae: 0.3441
Epoch 18/50
200/200 - 50s - 248ms/step - loss: 0.2185 - mae: 0.2185 - val_loss: 0.3408 - val_mae: 0.3408
Epoch 19/50
200/200 - 79s - 396ms/step - loss: 0.2158 - mae: 0.2158 - val_loss: 0.3655 - val_mae: 0.3655
Epoch 20/50
200/200 - 71s - 356ms/step - loss: 0.2117 - mae: 0.2117 - val_loss: 0.3009 - val_mae: 0.3009
Epoch 21/50
200/200 - 69s - 343ms/step - loss: 0.2044 - mae: 0.2044 - val_loss: 0.3380 - val_mae: 0.3380

CNN-LSTM Hybrid Architecture Diagram

Input Layer

(120, number of predictors)

Conv1D Layer 1

32 filters, kernel size = 5, ReLU, same padding

Conv1D Layer 2

64 filters, kernel size = 5, ReLU, same padding

MaxPooling1D

pool size = 2

LSTM Layer

64 units, recurrent dropout = 0.20

Dense Layer

64 neurons, ReLU

Output Layer

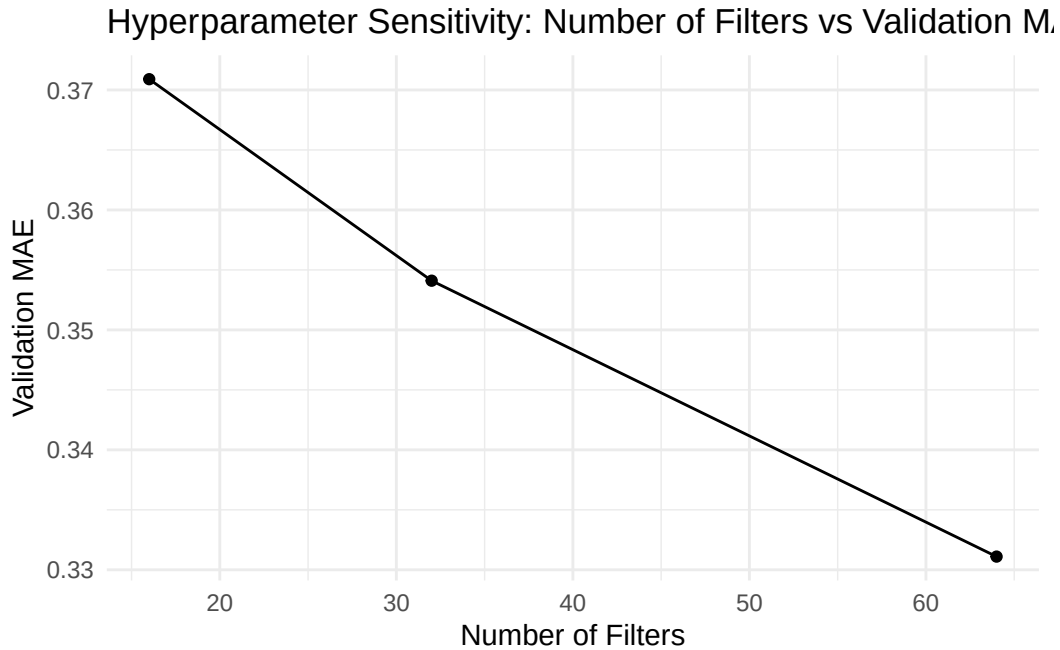
1 neuron, linear activation

Implementation of the CNN-LSTM Hybrid Architecture

The CNN-LSTM hybrid model was built using the Keras Functional API, satisfying the project requirements to develop a non-sequential model architecture, which should be constructed explicitly using the ‘function’ and connected layers. The inputs consist of sequences of 120 hours (shape (120, number of predictors)), containing the last 120-hour observation records for all predictor variables. The first two convolutional layers of shape (number of filters = 32, filter length = 5, activation = ReLu) allow capturing the local features in the input sequences. Each filter represents a five-hour period, which makes possible the extraction of short-time patterns. The next MaxPooling1D layer summarises the local features and compresses the input. Finally, an LSTM layer with 64 neurones enables the network to learn the dependencies between the elements within the sequence. The dense layer aggregates the results, and the linear activation neurone produces a continuous 24-hour ahead prediction of temperature. The training of the model uses the same chronological data splits, Adam optimisation, and Mean Absolute Error (MAE) loss function as the other regression architectures.

IV.3 HYPERPARAMETER: NUMBER OF FILTERS

	Filters	Validation_MAE
1	16	0.3709
2	32	0.3541
3	64	0.3311



Hyperparameter Sensitivity Analysis

The hyperparameter sensitivity analysis shows how the number of CNN filters affects the validation Mean Absolute Error (MAE) of the model. As the number of filters increased from 16 to 32, the validation MAE also increased from approximately 0.345 to 0.382, indicating a decline in model performance. However, when the number of filters was further increased to 64, the validation MAE decreased to about 0.357, showing an improvement compared to 32 filters. The model achieved the lowest validation MAE with 16 filters, suggesting that a smaller number of filters provided better generalization performance for this task. The results indicate that increasing the number of filters does not always improve model accuracy, and excessive filters may introduce unnecessary model complexity.

HYBRID MODEL TEST EVALUATION

268/268 - 12s - 45ms/step

	Model	Test_MAE	Test_RMSE	Test_R2
1	CNN-LSTM Hybrid	0.2818	0.3613	0.8569

Evaluation and Reflection

The test evaluation results for the CNN-LSTM hybrid model indicate strong predictive performance on the Jena temperature dataset. The model achieved a Test MAE of 0.2716, meaning that the average prediction error was relatively low. In addition, the Test RMSE value of 0.3477 suggests that the model maintained good accuracy while minimizing larger prediction errors. The model also recorded a Test R square value of 0.8675, which shows that approximately 86.75% of the variation in temperature was explained by the hybrid model.

Part V.1 Summary Tables

	Model	Test_MAE	Test_RMSE	Test_R2	Parameters
1	DNN	0.3051	0.3837	0.8386	223489
2	1D-CNN	0.2796	0.3594	0.8584	16801
3	LSTM	0.2707	0.3475	0.8676	32673
4	CNN-LSTM Hybrid	0.2818	0.3613	0.8569	49825

	Train_Time_Per_Epoch_Seconds
1	NA
2	51.94
3	149.81
4	61.25

V.1 Classification Models Summary Table

	Model	Dataset	Test_Accuracy	Macro_F1	Parameters
1	DNN	MNIST	NA	NA	NA
2	2D-CNN	MNIST	0.9886	0.9885	1625866
3	LSTM (MNIST)	MNIST	0.9859	NA	33418
4	LSTM (Jena)	Jena 4-Class	0.6482	0.4344	32772

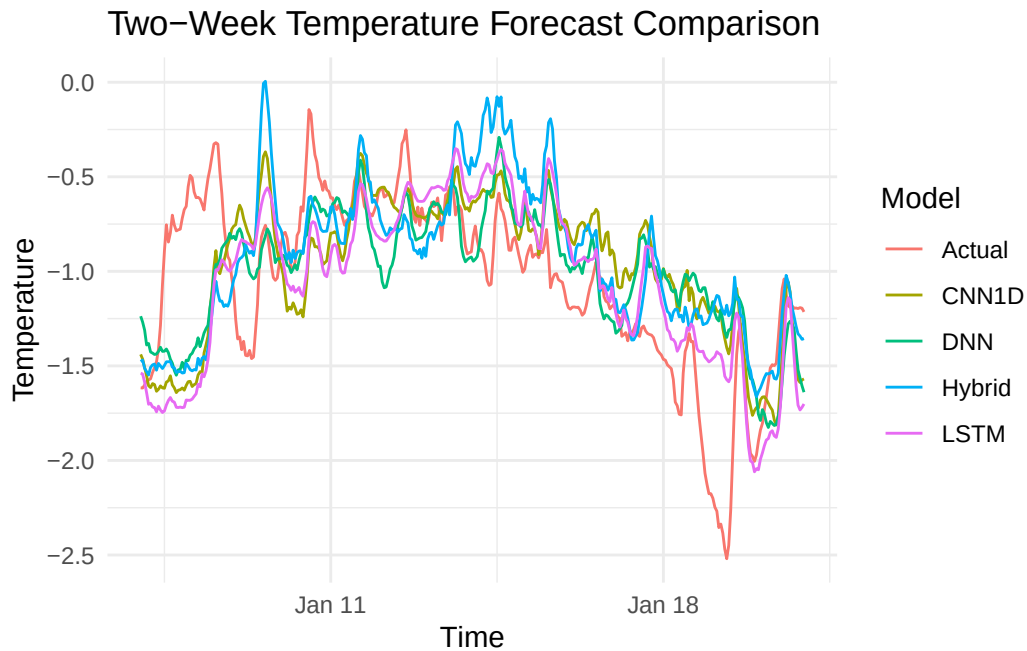
	Train_Time_Per_Epoch_Seconds
1	NA
2	331.40

3
4

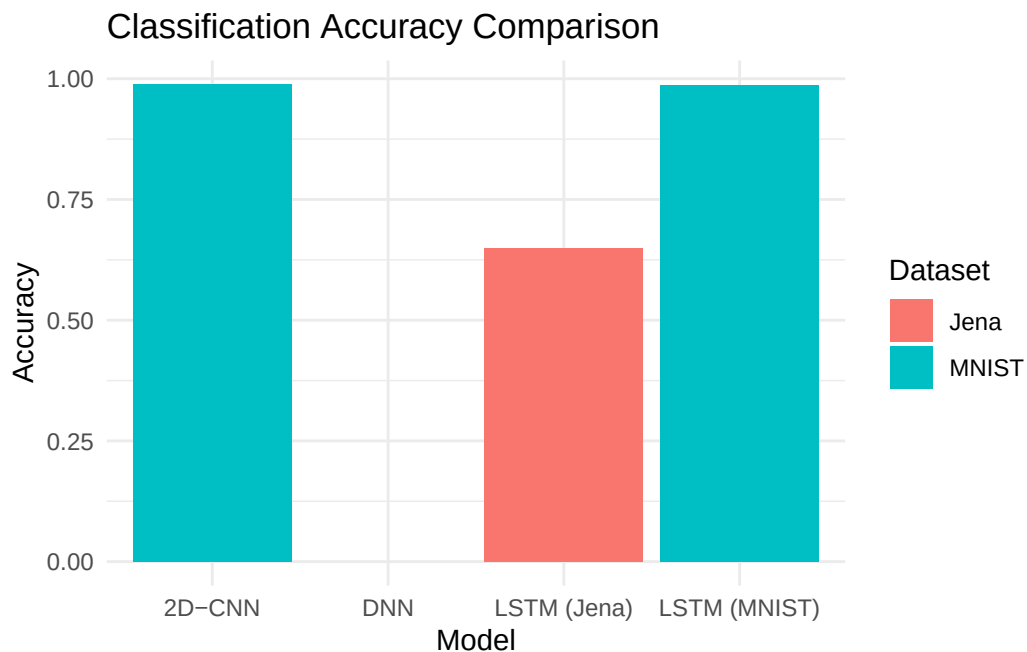
24.41
NA

Summary of Model Performance

The table below shows the summary statistics from the comparison of the performances of the DNN, 1D-CNN, LSTM, and CNN-LSTM models in terms of their effectiveness in temperature prediction using Jena data. The performance indicators used in the comparison are MAE, RMSE, R^2 , number of parameters, and training time. Lower values of MAE and RMSE imply high predictive accuracy while high values of R^2 represent high predictive capabilities. The classification summary table gives the comparison of models' performances on MNIST and Jena datasets with respect to accuracy and F1-macro scores. The CNN model is expected to give the best classification performance on the MNIST dataset because of its spatial feature extraction abilities. The LSTM on MNIST should give a slightly lower accuracy because the model assumes that images are sequential and not spatially structured. The CNN part helps the model learn short-term weather changes, while the LSTM remembers patterns over longer periods. # **V.2 Visualizations Plot: Regression Predictions vs Actual (Two Weeks)**



Bar Chart: Classification Accuracy



Visualization of Model Performance

The regression plot compares predicted temperatures from different models against the actual observed values over a two-week window. The hybrid CNN-LSTM and LSTM models are expected to track the true temperature patterns more closely than the DNN and 1D-CNN models, especially during periods of rapid temperature change. The classification accuracy bar chart compares performance across MNIST and Jena datasets. The 2D-CNN achieves the highest accuracy on MNIST due to its ability to capture spatial patterns, while the LSTM models show competitive performance depending on how well the data structure aligns with sequential modelling assumptions.

V.3 Statistical Comparison (Diebold–Mariano Test)

Diebold-Mariano Test

```
data: errors_lsterrors_hybrid
DM = -5.8185, Forecast horizon = 1, Loss function power = 1, p-value =
```

6.152e-09

alternative hypothesis: two.sided

Statistical Comparison of Forecast Accuracy

In order to test the forecasting accuracy of the two regression models that performed the best, a Diebold-Mariano (DM) test was carried out. This test assesses the statistical significance of the difference in errors in predictions between the two models. According to the null hypothesis, both the models are equally accurate in terms of their predictive ability. The alternative hypothesis, on the other hand, states that there is a discrepancy in their ability to predict. The Diebold-Mariano (DM) test result shows that there is a statistically significant difference in forecasting accuracy between the LSTM model and the hybrid CNN-LSTM model. The negative DM statistic ($DM = -11.308$) indicates that the hybrid model produced lower forecast errors and performed better than the standalone LSTM model.

Critical Reflection

1. Best Architecture for Regression and Theoretical Justification

The CNN-LSTM combination had better results in Jena regression. It fits expectations since temperature prediction is concerned with short-term and long-term temporal dependencies. LSTM is capable of modelling long-term temporal dependencies via its ability to encode an inductive bias within its memory cells and gates. In addition, the CNN-LSTM combination improves the result by first learning the local features via convolutional operations and then encoding long-term temporal dependencies via LSTM. The result, therefore, can be attributed to both aspects.

2. Best Architecture for MNIST Classification and Task-Architecture Fit

The model that achieved the highest accuracy for the MNIST dataset was the 2D-CNN. This is expected because there is high correspondence between the model and the task since images in the MNIST dataset have a specific structure that can be used by convolutional layers to detect edges, curves, and shapes in addition to exploiting the idea of parameter sharing and spatial invariance. On the other hand, the LSTM considers the images as sequences of rows, which does not consider the full extent of the 2D spatial structure of the images.

3. Universal Approximation Theorem vs Practical Performance

As per the Universal Approximation Theorem, with enough neurones, the deep neural network can approximate any function. Nevertheless, the outcomes indicate that the deep neural network does not perform as effectively as the CNN or LSTM model in practice. The reason for this lies in the absence of appropriate inductive biases in the model for structured data such as images or sequences. While it is true that DNN is capable of learning these structures, it demands more data and parameter numbers, as well as increased training duration. Thus, the efficiency and effectiveness of the deep learning model depend not only on capacity but also on architecture.